Project Interim Report

On

File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming With Java

SESSION: 2025-26

Submitted By

Student name: Sahil Raj

Reg No. :- 11250

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with IBM

Under the Guidance of

Mr. Santanu Sasmal

IBM SME and EXPERT

SCHOOL OF COMPUTER SCIENCE AND E NGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Abstract

Abstract

In mathematics and computer science, calculating the area of different geometric shapes is a fundamental and commonly required task. This project presents the design and implementation of a Geometric Shape Area Calculator using abstraction and interfaces in Object-Oriented Programming (OOP) with Java.

The system defines a common contract through a Shape interface, which is implemented by concrete classes such as Circle and Square. Each class provides its own shape-specific area calculation logic. The project also demonstrates runtime polymorphism by using interface references to call overridden methods. New shapes can be added to the system without modifying any existing code, making the design highly extensible and maintainable.

Introduction

Introduction

This project builds a Geometric Shape Area Calculator using Core Java and OOP principles. It models real-world geometry using classes and interfaces, allowing users to calculate the area of different shapes through a unified interface.

Key features of the project include:

- A Shape interface that defines a common calculateArea() method for all shapes
- Circle and Square classes that each implement the Shape interface independently
- Demonstration of runtime polymorphism using Shape reference variables
- Extensible design — new shapes (Triangle, Rectangle) can be added without any changes to existing code
- Clean design principles including abstraction, encapsulation, and code modularity

Objectives & Scope

Objectives

- Define a Shape interface as an abstraction contract
- Implement Circle and Square area calculations
- Apply polymorphism via interface references
- Demonstrate method overriding in each class
- Apply OOP concepts: abstraction, inheritance, polymorphism
- Write extensible, modular, and readable Java code

Scope

- Suitable for academic OOP coursework and lab practicals
- Covers fundamental Java interface & class design patterns
- Can be extended with Triangle, Rectangle, Hexagon etc.
- Future scope: database integration, GUI with Swing/JavaFX
- Applicable to real-world geometry and measurement tools

System Requirements

System Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: At least 500 MB free disk space
- Display: 1024 × 768 resolution or higher

Software Requirements

- Operating System: Windows / Linux / macOS
- Java JDK 8 or higher (java.lang, Math library)
- IDE: VS Code / Eclipse / IntelliJ / Edit Plus
- Java Compiler: javac (included with JDK)

Methodology

Methodology

The project follows a structured OOP-based design methodology:

Step 1

Define Shape Interface

Create a Java interface Shape with an abstract method calculateArea() that all shape classes must implement.

Step 2

Create Circle Class

Implement Shape in Circle. Store radius as an attribute. Override calculateArea() using formula: $\pi \times radius^2$.

Step 3

Create Square Class

Implement Shape in Square. Store side as attribute. Override calculateArea() using formula: side × side.

Step 4

Apply Polymorphism

Use Shape reference (Shape s1 = new Circle(5)) to call calculateArea() — runtime dispatches to the correct class method.

Step 5

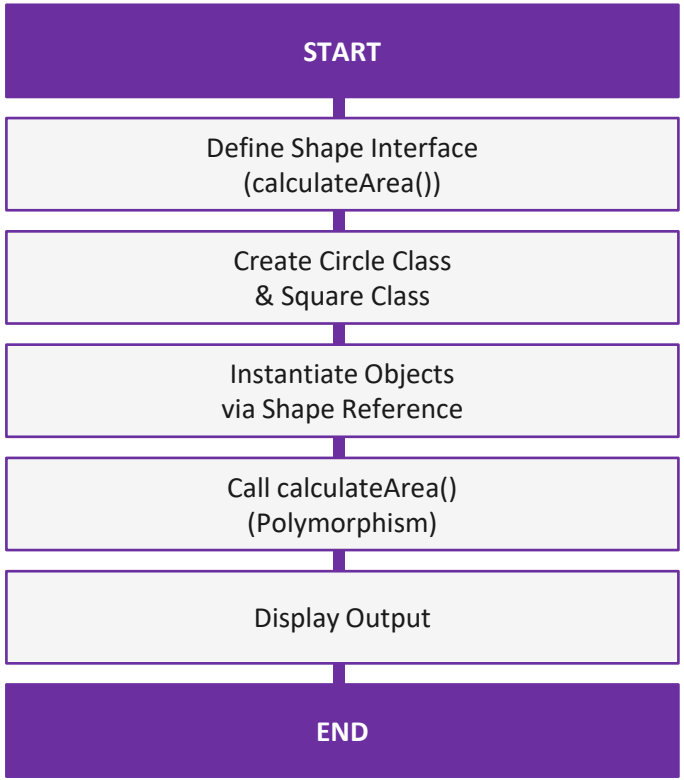
Test & Display Output

In Main class, create multiple shape objects, call calculateArea() and print results to console.

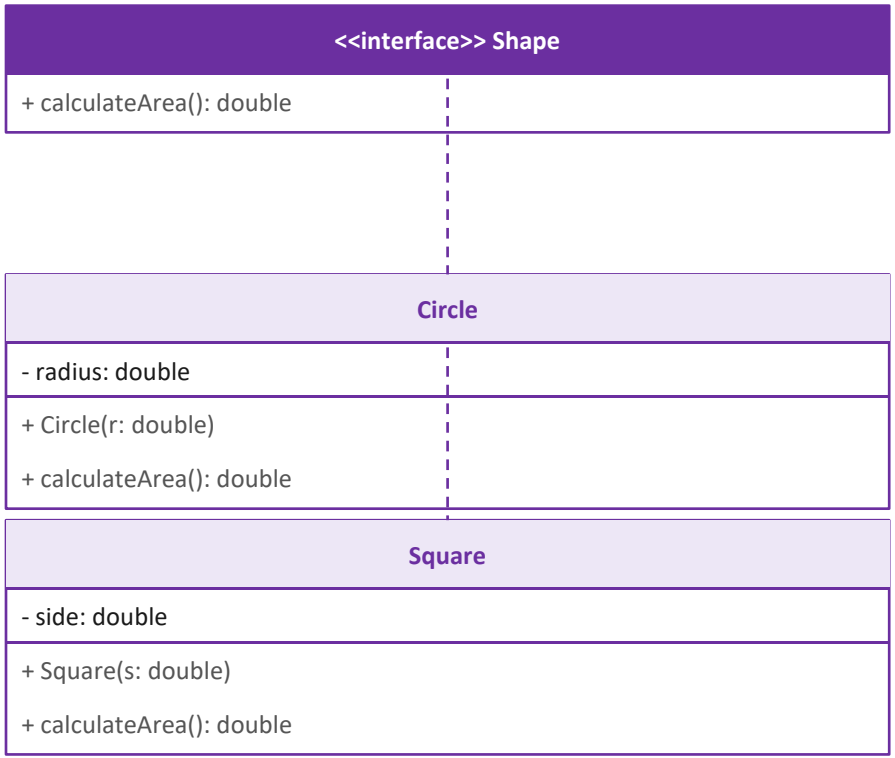
Flowchart

Flowchart

Program flow from start to output:



Class Diagram



Implementation

Implementation

The system is built using four logical components (code not pasted here — logic explained below):

Shape (Interface)

Declares the method calculateArea() as an abstract contract. Any class that implements Shape must provide its own area calculation logic.

Circle (Class)

Implements Shape. Stores radius as a private double. In calculateArea(), returns Math.PI × radius × radius. Constructor accepts radius.

Square (Class)

Implements Shape. Stores side as a private double. In calculateArea(), returns side × side. Constructor accepts the side length.

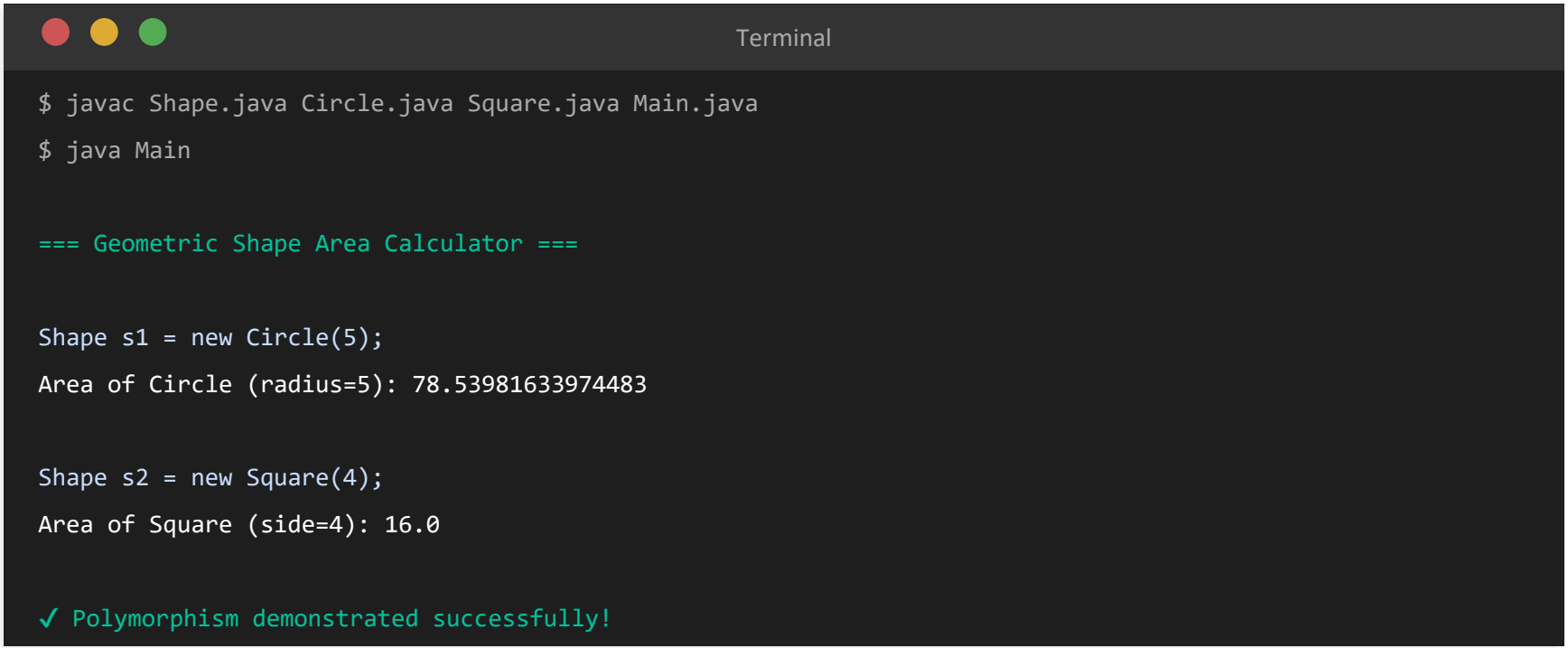
Main (Class)

Creates Shape references pointing to Circle and Square objects. Calls calculateArea() on each — runtime polymorphism selects the correct method and prints the result.

Sample Input / Output

Sample Input / Output

The program is run from the terminal. Below is a screenshot simulation of the expected console output:

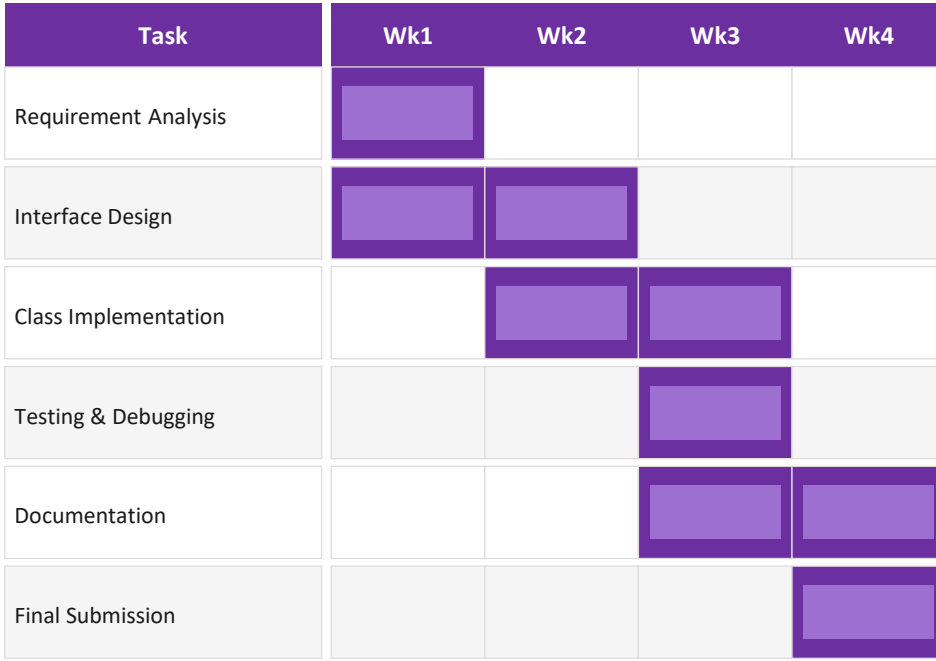


Future Enhancements & Gantt Chart

Future Enhancements

- Add more shapes: Triangle, Rectangle, Hexagon, Ellipse
- Build a GUI using Java Swing or JavaFX
- Integrate database (MySQL) to store shape records
- Add input validation and exception handling
- Support 3D shapes: Sphere, Cube, Cylinder
- Export results to PDF or Excel file

Gantt Chart (4 Weeks)



Conclusion

Conclusion

The Geometric Shape Area Calculator project successfully demonstrates the application of core Object-Oriented Programming concepts in Java. Through this project, the following outcomes were achieved:

- Abstraction was applied through the Shape interface, hiding implementation details from the caller
- Polymorphism was demonstrated by calling calculateArea() via Shape references at runtime
- Encapsulation was maintained by keeping attributes (radius, side) private in each class
- The design is extensible — new shape classes can be added with zero modification to existing code
- Clean design principles and code readability were followed throughout the implementation

The project provides a functional and well-structured system that serves as a strong foundation for further enhancement with GUI, database support, and additional shape types.

References

References

Book

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.

[2] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River: Prentice Hall, 2006.

Website

[3] Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com

[4] GeeksforGeeks, "Interfaces in Java," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org

Journal Article

[5] A. Sharma and R. Gupta, "OOP Design Patterns in Java," International Journal of CS, vol. 5, no. 2, pp. 45–50, 2020.

Conference Paper

[6] S. Kumar, "Teaching OOP with Real-World Projects," in Proc. IEEE Conf. on CS Education, 2021, pp. 120–125.